

Operating Systems Fundamentals

May 5, 2026

Disclaimer

Material and Slides

Some of the material/slides are adapted from various:

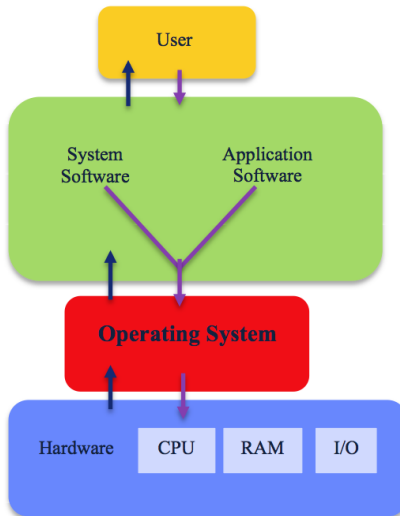
- Presentations found on the internet;
- Books;
- Web sites;
- ...

Outline

- 1 Operating System
- 2 Operating Systems Jargon
- 3 Types of Operating Systems
- 4 General vs Real Time Purpose
- 5 The Kernel
- 6 The Linux kernel
- 7 Execution Modes
- 8 Interfacing with hardware and software
- 9 Execution Context
- 10 Process Management
- 11 Memory Management
- 12 Dealing with time
- 13 Other Subsystems

- 1 Operating System**
- 2 Operating Systems Jargon
- 3 Types of Operating Systems
- 4 General vs Real Time Purpose
- 5 The Kernel
- 6 The Linux kernel
- 7 Execution Modes
- 8 Interfacing with hardware and software
- 9 Execution Context
- 10 Process Management
- 11 Memory Management
- 12 Dealing with time
- 13 Other Subsystems

Logical View



What is an Operating System (OS)?

- An OS **is the program that**, after being initially loaded into the computer by a boot program, **manages all of the software and hardware on the computer** (generally referred as **computer resources**).
 - OS is a program **that acts as an Interface between the system hardware and the user** making the tasks easier.
 - It performs basic tasks such as file, memory and process management, handling input and output, and controlling peripheral devices such as disk drives and printers.
 - The OS **coordinates all of this to make sure each program gets what it needs**.
 - Most of the time, there are several different programs (software) running at the same time, and they all need to access your computer's central processing unit (CPU), memory, storage and so on.
- The computer programs **make use of the OS by making requests for services through a defined application program interface (API)**.
 - Such API is, typically, implemented through **system calls**.

Why do we need an OS?

- **Convenience:**

- OSs enable users to get started on the things they wish to complete quickly without having to cope with the stress of first configuring the system.
- High-level **abstraction of physical resources**;

- **Efficiency:**

- An OS enables the efficient use of resources.
 - This is due to less time spent configuring the system.

- **Ability to evolve**

- An OS should be designed in such a way that it allows for the effective development, testing, and introduction of new features without interfering with service.
- Enable **portable code**.

- **Management of system resources**

- OS guarantees that resources are shared fairly among various processes and users.
 - Fairly?????? Depends on

Tasks of the Operating System (I)

- **Interface between the user and the computer:**
 - An OS provides a very easy way to interact with the computer.
- **Booting**
 - Booting is basically the process of starting the computer.
 - It helps to start the computer when the power is switched ON.
- **Managing the input/output devices**
 - It helps to operate the different input/output devices.
 - It decides which program or process can use which device.
 - It controls the allocation and deallocation of devices.
- **Platform for other application software**
 - Users require different application programs to perform specific tasks on the system.
 - IT manages and controls these applications so that they can work efficiently.
 - It loads and runs application software as well as system software.

Tasks of the Operating System (II)

- **Manages the memory**
 - It allocates and deallocates memory to all the applications/tasks.
- **Manages the system files**
 - It helps to manage files on the system.
 - All the data on the system is in the form of files.
 - It makes interaction with the files easy.
- **Provides Security**
 - The operating system provides various techniques which assure the integrity and confidentiality of user data.
 - It keeps the system and applications safe through authorization/permission.

The System and Application Programming

- **System components:**

- **System software**

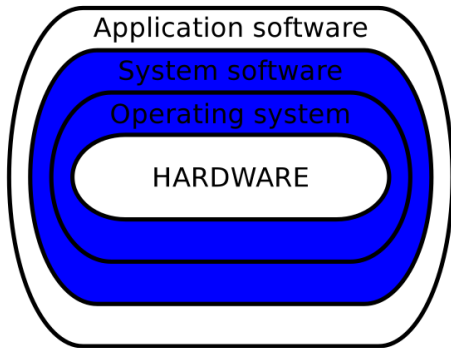
- Command Line Interface (Shell);
 - Graphical User Interface (GUI);
 - ...

- **Operating system**

- Process management;
 - Memory management;
 - ...

- **Application programming**

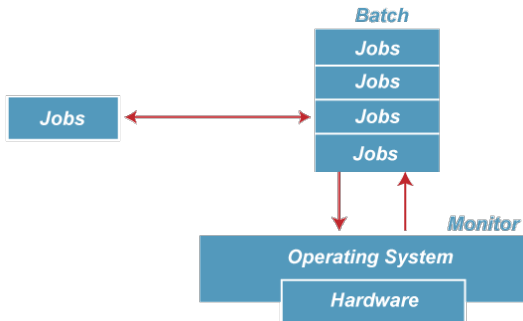
- There is a trend in application programming, away from system-level programming and towards very high-level (application) development;
 - There is no programmer, however, **who does not benefit from some understanding of system programming;**



- 1 Operating System
- 2 Operating Systems Jargon**
- 3 Types of Operating Systems
- 4 General vs Real Time Purpose
- 5 The Kernel
- 6 The Linux kernel
- 7 Execution Modes
- 8 Interfacing with hardware and software
- 9 Execution Context
- 10 Process Management
- 11 Memory Management
- 12 Dealing with time
- 13 Other Subsystems

Batch Processing

- The system put **all of the jobs in a queue on the basis of first come first serve and then executes the jobs one by one.**
 - Executes one task or activity at a time (single processing)
- The users **collect their respective output when all the jobs get executed.**

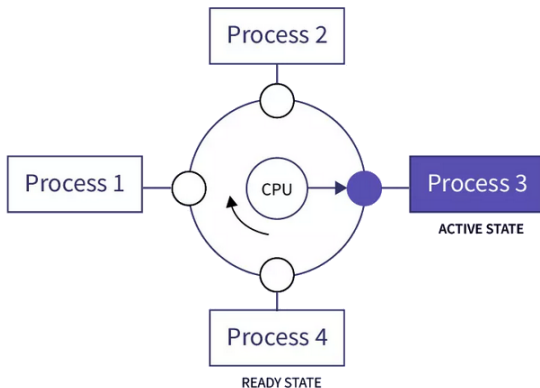


Multiprogramming

- Multiprogramming **is an extension to batch processing where the processor or Central Process Unit (CPU) is always kept busy.**
- Each program needs two types of **system time**:
 - CPU time
 - Input/output (I/O) time
- When a program is being performed, it is known as a **Task/Job** and **Process**.
- It is an execution strategy in which **multiple processes are loaded into memory**, with the **processes taking turns executing instructions on the CPU** for a short time or according to any other heuristic.
 - **Concurrent program executions improve system resource utilization** as compared to batch processing systems.

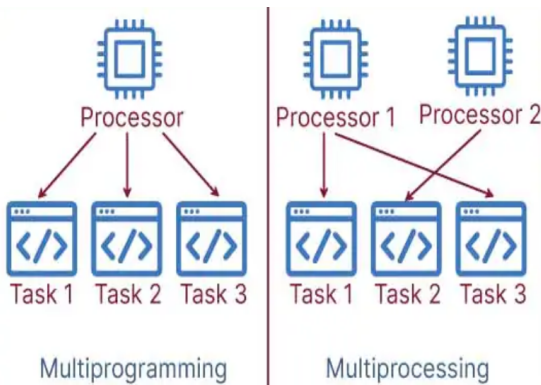
Time-Sharing

- It uses the concept of CPU scheduling and multiprogramming so that the **time of the CPU is divided** equally (typically) among the running tasks/processes (or users).



Multiprocessing

- In Multiprocessing, **Parallel computing is achieved**.
- There are more than one processors present in the system which can execute more than one process at the same time.
 - Note, multicore systems have multiple processors in a single package



Asymmetric vs Symmetric Multiprocessing (I)

- **Asymmetric Multiprocessing:**

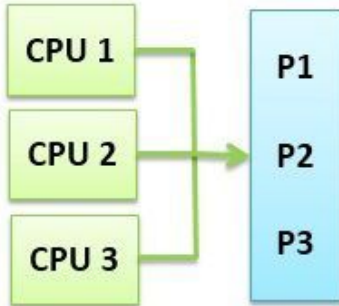
- It is a multiprocessor computer system where not all of the multiple interconnected CPUs are treated equally.
- In asymmetric multiprocessing, only **a master processor runs the tasks of the operating system.**

- **Symmetric Multiprocessing:**

- It involves a multiprocessor computer hardware and software architecture where two or more identical processors are connected to a single, shared main memory, and have full access to all input and output devices.
- In other words, Symmetric Multiprocessing is a type of multiprocessing where **each processor is self-scheduling.**

Asymmetric vs Symmetric Multiprocessing (II)

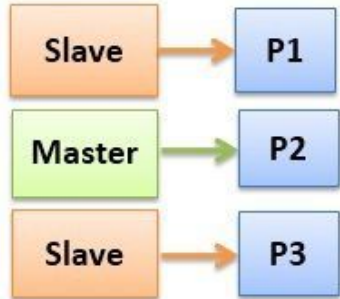
Symmetric Multiprocessing



(Shared Memory)

Vs

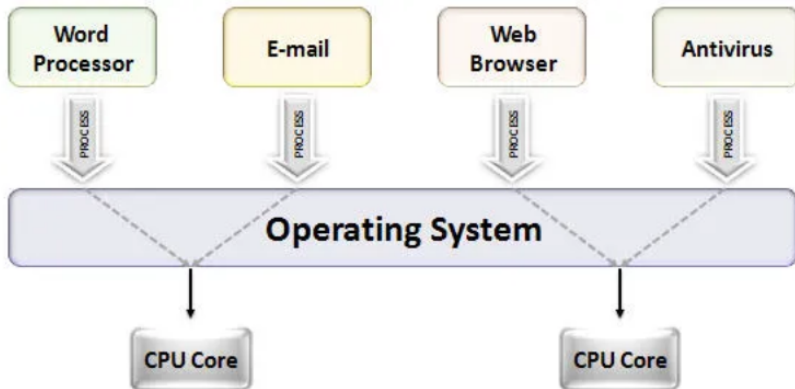
Asymmetric Multiprocessing



(No Shared Memory)

Multitasking

- The multitasking operating system **is a logical extension of a multiprogramming system** that enables multiple programs simultaneously.
- Multiprocessing **also enables multitasking.**



- 1 Operating System
- 2 Operating Systems Jargon
- 3 Types of Operating Systems**
- 4 General vs Real Time Purpose
- 5 The Kernel
- 6 The Linux kernel
- 7 Execution Modes
- 8 Interfacing with hardware and software
- 9 Execution Context
- 10 Process Management
- 11 Memory Management
- 12 Dealing with time
- 13 Other Subsystems

Types (I)

- **Desktop Operating System**
 - Windows, Linux, and MacOS are the most popular.
- **Server Operating System**
 - Linux or UNIX is more preferred for servers
- **Mobile Operating System**
 - Android and iOS
- **Supercomputer Operating Systems**
 - Supercomputers are used in weather forecasting, space research and wherever a huge amount of data needs to be parallel processed in a short amount of time
- **Embedded, Industrial Robots and Controllers Operating Systems**
 - VxWorks, RTLinux, Lynx, QNX, RTEMS, Zephyr. FreeRTOS, and ...

Types (II)

- **Distributed Operating System**

- This system is based on autonomous but interconnected computers communicating with each other via communication lines or a shared network.
- Each autonomous system has its own processor that may differ in size and function.
- These operating systems are often used for tasks such as telecommunication networks, airline reservation controls and peer-to-peer networks.

- **Cloud based Operating System**

- These are lightweight operating systems used for cloud based use cases.
- Chrome OS (linux based and uses google chrome as interface), EasyPeasy (debian and unbuntu) linux based OS for netbooks), Joli OS (Ubuntu Linux based), EyeOS (gives desktop interface within web page), OSW3, DOKY (linux based)

Types (III)

- **Network Operating System**

- Network operating systems are installed on a server, providing users with the capability to manage data, user groups and applications.
- This operating system enables users to access and share files and devices such as printers, security software and other applications, mainly in a local area network.

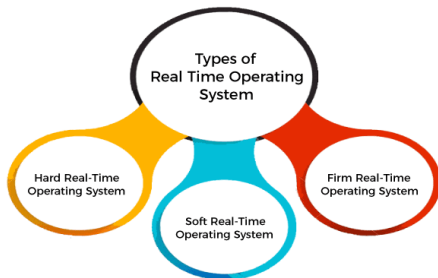
- 1 Operating System
- 2 Operating Systems Jargon
- 3 Types of Operating Systems
- 4 General vs Real Time Purpose**
- 5 The Kernel
- 6 The Linux kernel
- 7 Execution Modes
- 8 Interfacing with hardware and software
- 9 Execution Context
- 10 Process Management
- 11 Memory Management
- 12 Dealing with time
- 13 Other Subsystems

General-Purpose Operating System (GPOS)

- The GPOS is an operating system that can manage a high number of processes and complete an execution per unit time (defined as **throughput**).
- GPOS use a **fairness policy** to carry processes and threads to the CPU.
 - The task scheduler uses a fairness policy, allowing the overall high throughput but not ensuring that high-priority jobs will be executed first.
- GPOS is designed to perform non-time-critical general tasks.

Real-Time Operating System (RTOS)

- A RTOS is a type of operating system that is designed to meet strict time constraints, with a guaranteed response time for critical tasks.
- In RTOS, each job/task/process carries a certain **deadline within which the job is supposed to be completed**, otherwise, the huge loss will be there, or even if the result is produced, it will be completely useless.



Types of RTOS

- **Hard**

- Hard real-time is when a system **will cease to function if a task deadline is missed**, which can result in catastrophic consequences.
- These systems typically require deterministic and predictable behavior, with a guaranteed response time for critical tasks.

- **Soft**

- Soft real time system is a system that **will not cease to function if a task deadline is missed**.
 - Result obtained **after deadline is not incorrect**.
 - Its operation is degraded if results are not produced according to the specified timing requirement.

- **Firm**

- Firm real-time is a **system that will not cease to function if a task deadline is missed**, but the result of such task is discarded.
 - The utility of result becomes zero after the deadline.
 - Result obtained **after deadline is considered incorrect**.

- 1 Operating System
- 2 Operating Systems Jargon
- 3 Types of Operating Systems
- 4 General vs Real Time Purpose
- 5 The Kernel**
- 6 The Linux kernel
- 7 Execution Modes
- 8 Interfacing with hardware and software
- 9 Execution Context
- 10 Process Management
- 11 Memory Management
- 12 Dealing with time
- 13 Other Subsystems

What is a Kernel in OS?

- Kernel is the **core of the OS**.
 - It provides basic services for all other parts of the OS.
 - It is the main layer between the OS and underlying computer hardware, and it helps with tasks such as process and memory management, file systems, device control and networking.
- During normal system startup, a computer's basic input/output system, or BIOS, completes a hardware bootstrap or initialization.
- Then the bootloader loads the kernel from a storage device – such as a hard drive – into a protected memory space.
- Once the kernel is loaded into computer memory, the BIOS transfers control to the kernel.
- Finally, it then loads other OS components to complete the system startup and make control available to users through a desktop or other user interface.

Kernel Architecture: Monolithic

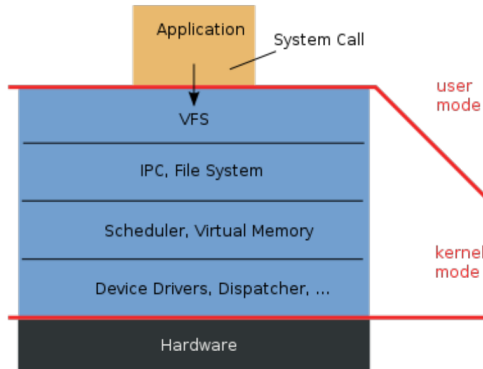
- In Monolithic kernel mode, OS runs in a **single address space**.
- Monolithic kernel **has all the OS functions or services** within a single kernel.
- Monolithic kernel **runs as a single process in a single address space in memory**.
- Monolithic architecture has two modes: **Kernel mode** and **User mode**.
 - Kernel mode has all the primary or core operation system features
 - User mode has the operating system features not added in the kernel mode
- Monolithic architecture enables higher performance however less flexible for modifications to add new features or enhance existing features.
 - Linux, Windows, MacOS

Kernel Architecture: Microkernel

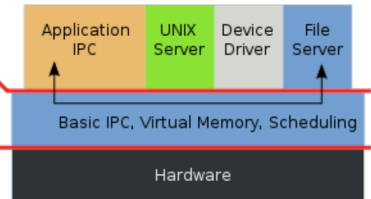
- Microkernel architecture is an architecture with **kernel having the basic interaction with hardware** and the basic Inter-Process Communication (IPC) mechanisms.
- In the microkernel architecture, Kernel **has the least amount of operating system core services**.
 - IPC, scheduling and memory management are the core services in a micro-kernel.
 - All the other OS services exist outside the Kernel.
 - It is broken down into several processes, known as **servers**;
 - All servers are kept separate and run in different address spaces;
- Microkernel provides the flexibilities to add new features or modify existing features while slightly affecting performance as it increases amount of interactions between kernel and user mode features.
 - QNX

Monolithic vs Microkernel

Monolithic Kernel based Operating System



Microkernel based Operating System

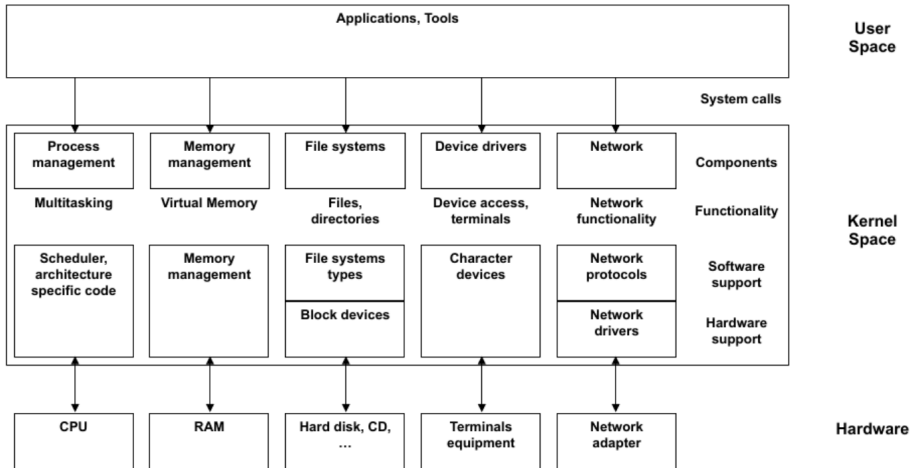


- 1 Operating System
- 2 Operating Systems Jargon
- 3 Types of Operating Systems
- 4 General vs Real Time Purpose
- 5 The Kernel
- 6 The Linux kernel**
- 7 Execution Modes
- 8 Interfacing with hardware and software
- 9 Execution Context
- 10 Process Management
- 11 Memory Management
- 12 Dealing with time
- 13 Other Subsystems

Overview

- Linux is a **monolithic kernel**.
 - Executes in a **single address space** entirely in kernel mode.
- However, **borrow**s much of the good from microkernels.
 - Boasts a modular design: Linux **supports the dynamic loading of kernel modules**.
 - It can dynamically load and unload kernel code on demand.
- The Linux kernel **is preemptive**.
 - It can preempt a task even as it executes in the kernel;
- Linux takes an interesting approach to thread support: It does not differentiate **between threads and normal processes**.
- Support for kernel threads;
- Execution modes
 - **Kernel mode and User mode**
- Execution contexts
 - **Kernel-space and User-space**

Structure (I)



Structure (II)

- Components **can be enabled or disabled at compile time.**
- It follows a **layered architecture.**
 - User space, Kernel space, and hardware
- Organize the kernel in **logical and independent subsystems** (components).
- Strict interfaces between layers.
 - System calls **for interfacing User and Kernel space.**
 - Device drivers and others (typically, architecture-dependent code) **for interfacing Kernel space and hardware.**

- 1 Operating System
- 2 Operating Systems Jargon
- 3 Types of Operating Systems
- 4 General vs Real Time Purpose
- 5 The Kernel
- 6 The Linux kernel
- 7 Execution Modes**
- 8 Interfacing with hardware and software
- 9 Execution Context
- 10 Process Management
- 11 Memory Management
- 12 Dealing with time
- 13 Other Subsystems

The Kernel

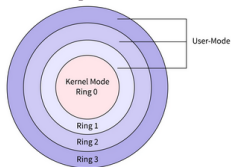
- Kernel **is the central module of the operating system** and it works as the middle layer between the operating system and the hardware of a system.
 - Operating systems maintain control over the computer system by utilizing the kernel of an operating system as a means of communication.
- The kernel handles the most system functions on behalf of the operating system, hence it is the first software to load into memory when a system boots up after the `bootloader`.
- The kernel **is in the memory of the system until the operating system shuts down the system.**

Kernel mode vs User mode (I)

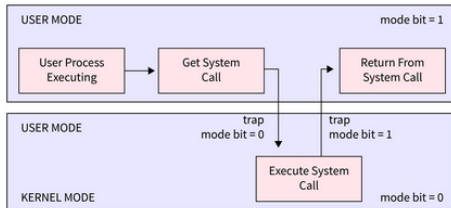
- Whenever you run an application in your computer system, it runs in the **user mode**.
- Accessing to the hardware in user mode requires sending a request to the kernel.
 - That is the reason user mode is also known as slave mode, or **restricted mode**.
- **Kernel mode** exists to keep user applications from disrupting critical system functions.
- The kernel can access the hardware of the system directly.
- There are **some privileged instructions that can run in the kernel mode only**.
 - These instructions are interrupt instructions, input-output management, etc.

Kernel mode vs User mode (II)

- **Operating system protection rings** are the most common way to implement a user mode apart from kernel mode.



- In kernel mode, the mode bit is set to 0.
- And when switching from kernel mode to user mode, the bit mode is changed from 0 to 1.



Kernel mode vs User mode (III)

Criteria	Kernel Mode	User Mode
Resource Access	There is direct access to the hardware.	There is no direct access to the hardware. To access the hardware, there is a need to make a system call.
Virtual Address Space	All the processes and requests share the same virtual address space.	All the processes and requests share a separate virtual address space.
Restrictions	There are no restrictions.	There are restrictions.
Functionality	The kernel mode is able to refer to any memory block in the computer system and it can access the resources directly to execute a process that makes this mode a very potent and important mode for the users.	This is a very basic and standard viewing mode. It means that this mode can not access the hardware that makes it to execute the process on its own. It uses an Application Protocol Interface (API), provided by kernel, for this purpose.

- 1 Operating System
- 2 Operating Systems Jargon
- 3 Types of Operating Systems
- 4 General vs Real Time Purpose
- 5 The Kernel
- 6 The Linux kernel
- 7 Execution Modes
- 8 Interfacing with hardware and software**
- 9 Execution Context
- 10 Process Management
- 11 Memory Management
- 12 Dealing with time
- 13 Other Subsystems

Managing hardware

- How to deal **with different processing/response speeds**?
 - For instance, CPUs are much faster than other hardware components;
 - **It is not ideal for the kernel to issue a request and wait for a response** from the significantly slower hardware;
 - The kernel must **be free to go and handle other work, dealing with the hardware only after that hardware has actually completed its work.**
- How can the CPU work with hardware without impacting the machine's overall performance?
 - **Polling:**
 - **Periodically**, the kernel can check the status of the hardware in the system and respond accordingly.
 - **Interrupt:**
 - A mechanism for the hardware **to signal** to the kernel when attention is needed.

Interrupts

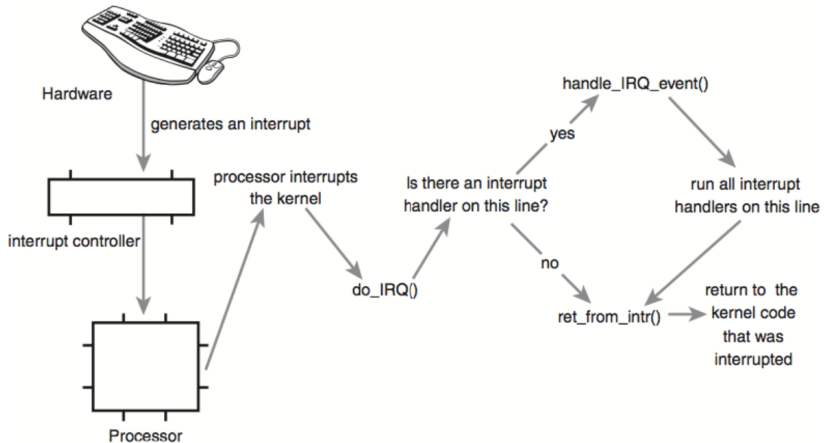
- An interrupt is usually **defined as an event that alters the sequence of instructions executed by a CPU**.
 - This literally interrupts the current process, which in turn interrupts the kernel.
- Interrupts are often divided into synchronous and asynchronous interrupts:
 - Synchronous interrupts **are produced by the CPU** control unit while executing instructions and are called synchronous because the control unit issues them only after terminating the execution of an instruction;
 - Asynchronous interrupts **are generated by other hardware devices** at arbitrary times with respect to the CPU clock signals.

Notice

Intel microprocessor manuals designate synchronous and asynchronous interrupts as **exceptions** and **interrupts**, respectively.

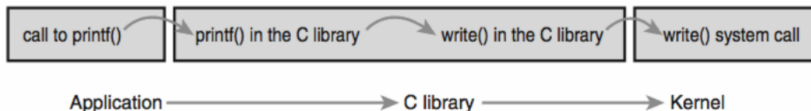
Interrupt handler

- The kernel uses the **interrupt's number** to execute a specific interrupt handler.



System Calls

- Applications typically call functions in libraries.
 - An intermediate layer to standardise and simplify the management of kernel routines across different architectures and systems.
- Libraries, in turn, rely on a **system call interface** to instruct the kernel to carry out tasks on the application's behalf .



- System calls are APIs for the interface between the user space and the kernel space.

System Calls: x86

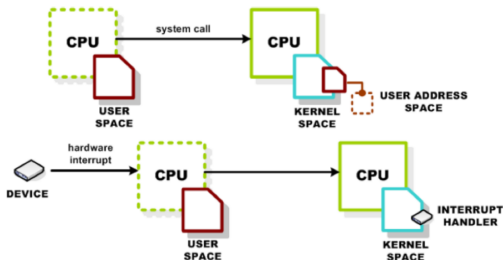
- Steps for invoking a system call:
 - 1 Put the system call number in the EAX register.
 - 2 Store the arguments to the system call in the registers EBX, ECX, etc.
 - 3 Call the relevant interrupt (80h).
 - 4 The result is usually returned in the EAX register.

```
mov edx,4 ; message length
mov ecx,msg ; message to write
mov ebx,1 ; file descriptor (stdout)
mov eax,4 ; system call number (sys_write)
int 0x80 ; call kernel
```

- 1 Operating System
- 2 Operating Systems Jargon
- 3 Types of Operating Systems
- 4 General vs Real Time Purpose
- 5 The Kernel
- 6 The Linux kernel
- 7 Execution Modes
- 8 Interfacing with hardware and software
- 9 Execution Context**
- 10 Process Management
- 11 Memory Management
- 12 Dealing with time
- 13 Other Subsystems

Kernel space vs User space

- Linux is always in one of three states at a given moment:
 - Process context** - in user-space.
 - When it is executing user code in a process (even when idle).
 - Process context** - in kernel-space.
 - When it is executing on behalf of a specific process.
 - Kernel code is running in the context of the process that invoked the system call
 - Interrupt context** - in kernel-space.
 - When it is executing an interrupt handler.



Kernel-space contexts

- In **process context**:
 - The kernel is capable **of sleeping** (for example, if the system call blocks on a call or explicitly calls `schedule()` - a kernel function);
 - The kernel is fully preemptible.
- In **interrupt context**:
 - It is not associated to any process;
 - It is lighter than a process (it has less context and require less time to set up or tear down);
 - This special context is occasionally called **atomic context** **because code executing in this context is unable to block.**

- 1 Operating System
- 2 Operating Systems Jargon
- 3 Types of Operating Systems
- 4 General vs Real Time Purpose
- 5 The Kernel
- 6 The Linux kernel
- 7 Execution Modes
- 8 Interfacing with hardware and software
- 9 Execution Context
- 10 Process Management**
- 11 Memory Management
- 12 Dealing with time
- 13 Other Subsystems

Overview

- It is multiuser because it allows **multiple users to use a system** and not affect each other's "stuff" (files, preferences, etc.).
- It is multitasking – the ability to perform **several operations** in parallel (concurrent).
 - The **switching intervals are so short**, users **do not notice the intervening brief periods of inactivity** and **gain the impression that the system is actually doing several things at once**.
- This kind of process management gives rise to several issues:
 - Processes **must not interfere with each other** unless this is expressly desired.
 - CPU **time must be shared** as fairly as possible between the various applications.

Processes

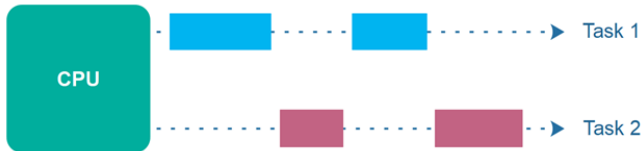
The Linux kernel data structures used to represent individual processes have connections with nearly every subsystem of the kernel.

Process

- A process **is an active program and related resources**.
 - From the kernel's point of view, the purpose of a **process is to act as an entity to which system resources (CPU time, memory, etc.) are allocated**.
- It may have one or more threads of execution
 - Each thread includes a unique program counter, process stack, and set of processor registers
- Provides two virtualisations, **giving the illusion that it alone monopolises the system**.
 - **Virtualised processor.**
 - **Virtualised memory.**

Virtualised processor

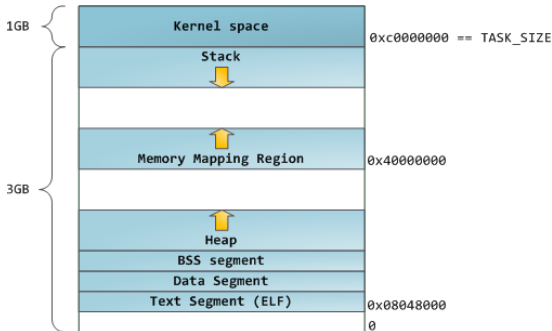
- Multiple processes **can run concurrently on the same system, and each process appears to have exclusive use of the hardware**, such as CPU.



- The instructions of **one process are interleaved with the instructions of another process.**
- Generally, **there are more processes to run than there are CPUs to run them.**
 - The OS performs this interleaving with a mechanism known as **context switching.**

Virtualised Memory

- Each process **runs in its own memory sandbox (virtual memory)**.
- Virtual memory is **an abstraction that provides each process with the illusion that it has exclusive use of the main memory**.

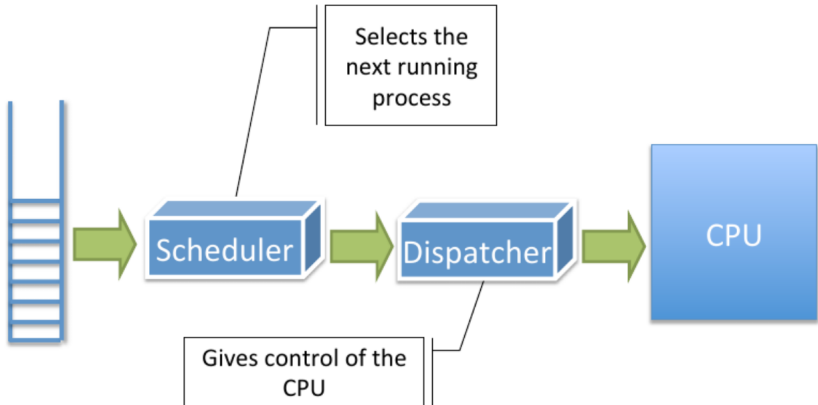


- **OS Kernel code and data are always addressable**, ready to handle interrupts or system calls at any time.

Process scheduling

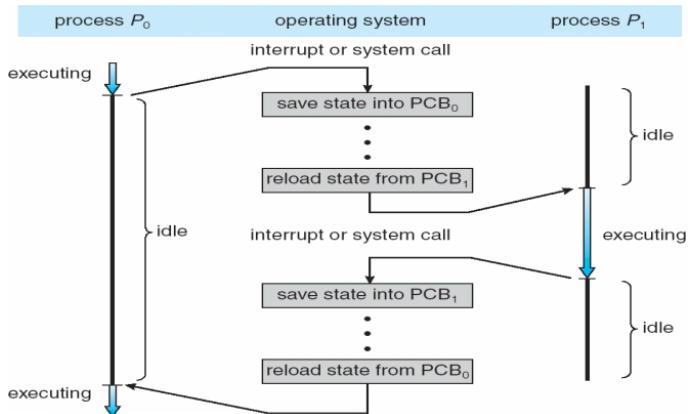
- The process scheduler **decides which process runs, when, and for how long**;
- By deciding which process runs next, the scheduler is **responsible for best utilizing the system**.
 - It divides the finite resource of CPU time between the runnable processes on a system
- It needs **a scheduling policy**, which defines the **scheduling rules**.
- The Dispatcher is the module that gives control of the CPU to the process selected by the scheduler
 - Switching context;
 - Switching to user space.
 - ...

Scheduler and dispatcher



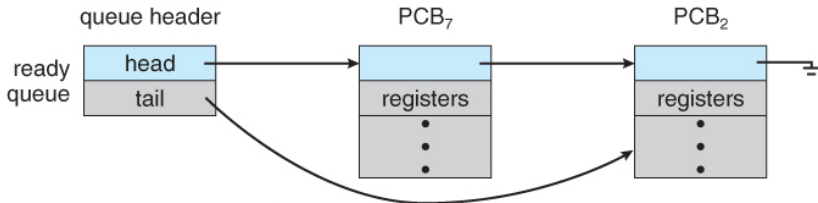
Context switching (I)

- The **act of switching from one process to another**.
- The system has to:
 - Save the **context of the current process**.
 - Restore the **context of the new process**.



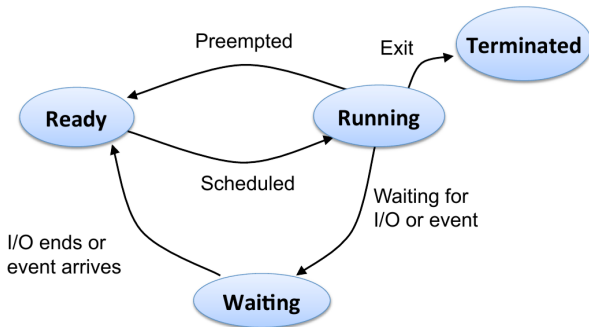
Context switching (II)

- What is the **context of a process**?
 - Program Counter
 - Stack Pointer
 - Registers
 - Code + Data + Stack (also called Address Space)
 - Other state information maintained by the OS for the process (open files, scheduling info, I/O devices being used, etc.)
- All this information is usually stored in a structure called **Process Control Block (PCB)**



Process State

- Multitasking implies process states.
 - Names for these states are not standardised, but they have similar functionality:
 - **Ready**: ready to run, but waiting to be scheduled;
 - **Running**: executing at the moment;
 - **Waiting**: waiting for I/O or an event;
 - **Terminated**: no longer ready to execute.



Scheduling events

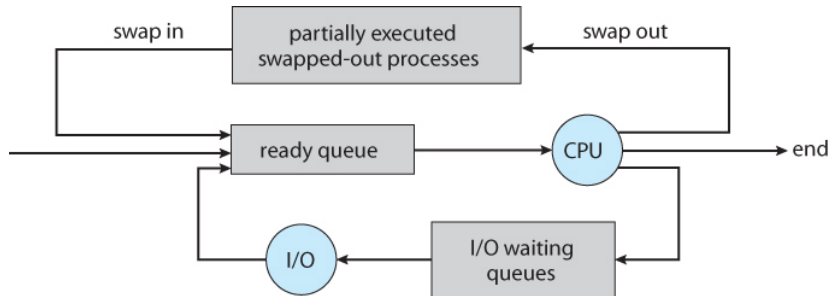
- A process switches **from the running state to waiting state** (e.g. I/O request);
- A process switches **from the running state to the ready state** (e.g. time slice expires);
- A process switches **from waiting state to ready state** (e.g. completion of an I/O operation);
- A process **terminates**

Notice

All executing tasks go from ready to running state. Note that, there is only one task in the running state, per CPU.

Scheduling queues

- **Ready queue:** All processes that are **ready for execution**
- **Wait queues:** When a process **is blocked** in an I/O operation or waiting for an event, it is put in a device/event queue



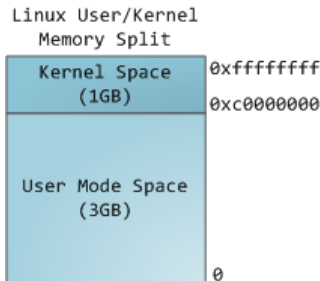
- 1 Operating System
- 2 Operating Systems Jargon
- 3 Types of Operating Systems
- 4 General vs Real Time Purpose
- 5 The Kernel
- 6 The Linux kernel
- 7 Execution Modes
- 8 Interfacing with hardware and software
- 9 Execution Context
- 10 Process Management
- 11 Memory Management**
- 12 Dealing with time
- 13 Other Subsystems

Memory Management

- The memory management subsystem is one of the most important parts of the operating system.
- Since the early days of computing, there has been a need for more memory than exists physically in a system.
- Strategies have been developed to overcome this limitation and the most successful of these is **virtual memory**.
- Virtual memory **makes the system appear to have more memory than it actually has** by sharing it between competing processes as they need it.

Process' Virtual Memory

- Each process in a multi-tasking OS runs in its **own memory sandbox**.
 - This sandbox is the **virtual address space**, which in 32-bit mode is always a 4GB block of memory addresses.
 - In Linux, **kernel space is constantly present and maps the same physical memory in all processes**.
 - **Kernel code and data are always addressable**, ready to handle interrupts or system calls at any time.
- These **virtual addresses are mapped to physical memory by page tables**, which are maintained by the operating system kernel and consulted by the processor.
 - Each process has its own set of page tables.

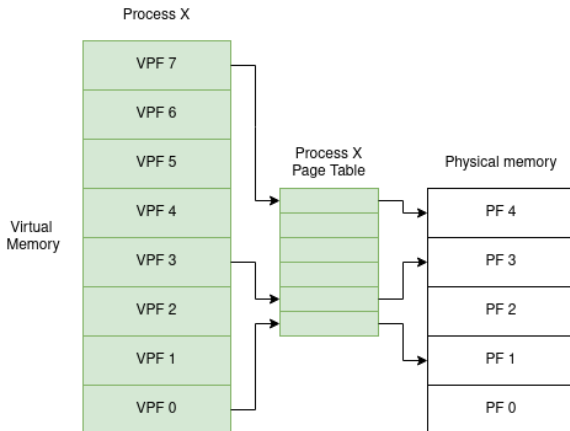


Mapping Virtual to Physical Memory (I)

- **Virtual and physical memory** are divided into chunks called **pages**.
- These pages are all the same size.
- Each of these pages is given a unique number;
 - The physical Page Frame Number (PFN).
 - The Virtual Page Frame Number (VPFN).
- **Page Table**
 - It is **a data structure used by the virtual memory system to store the mapping between logical addresses and physical addresses**.
 - Each process **owns its Page Table**.

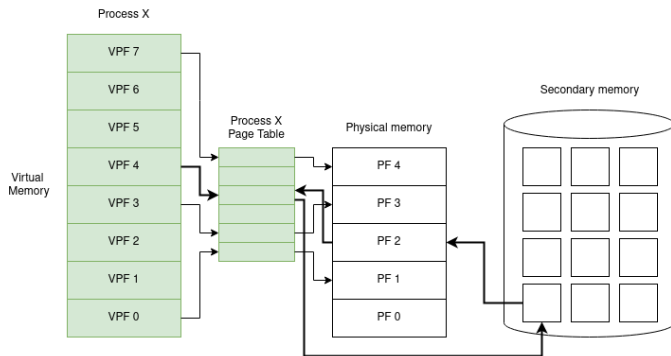
Mapping Virtual to Physical Memory (II)

- These **virtual addresses** are mapped to **physical memory** by **page tables**, which are maintained by the OS and consulted by the CPU.



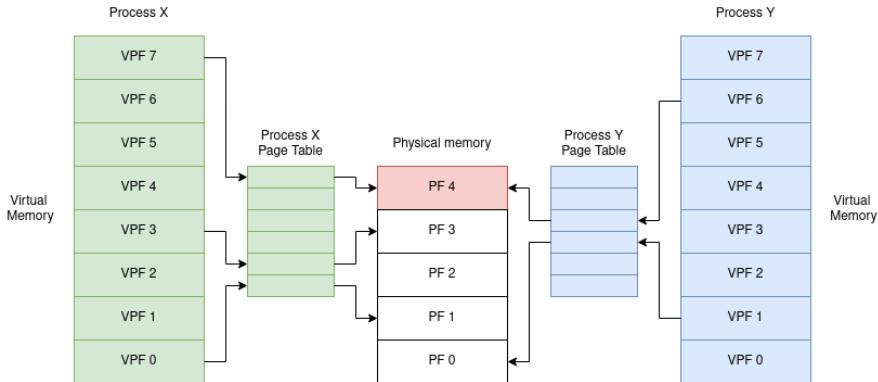
Mapping Virtual to Physical Memory (III)

- When a process needs to access a memory location that is not in physical memory, the **page table entry** for that location indicates that a **page fault** has occurred.
- The OS loads the required page into physical memory from disk, updating the page table accordingly.

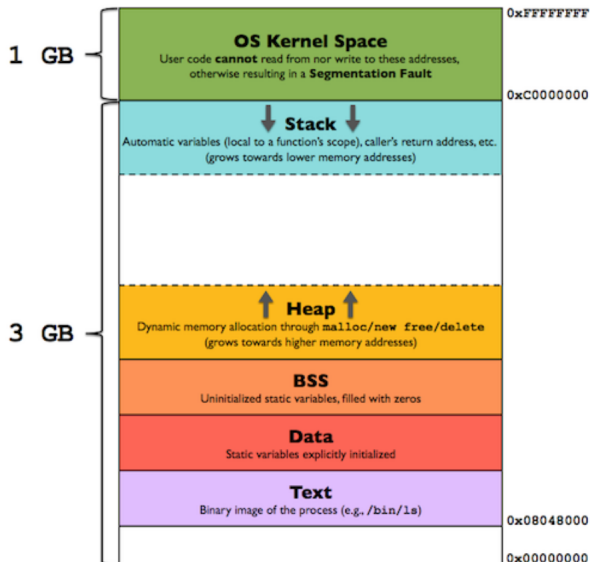


Mapping Virtual to Physical Memory (IV)

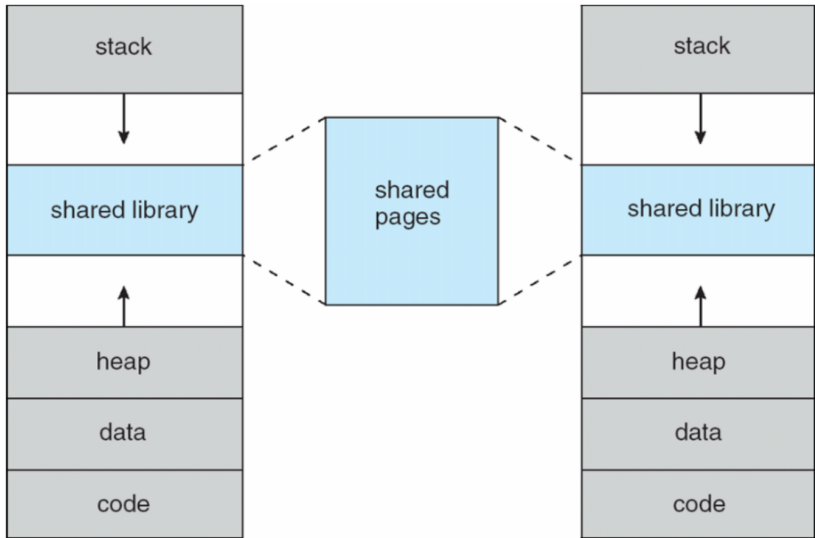
- Each process has its page table.
- This mechanism allows memory to be **shared across several processes**.



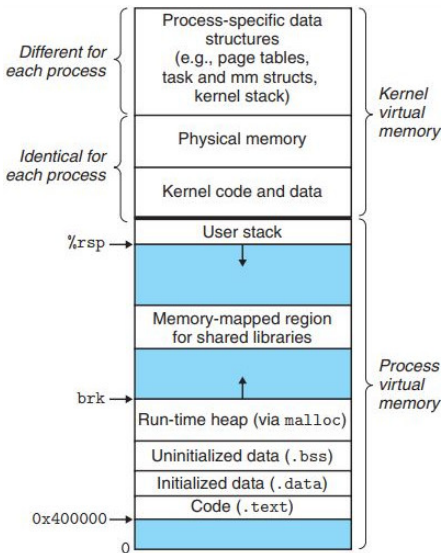
Process Memory Organization



Address Space Shared among Processes



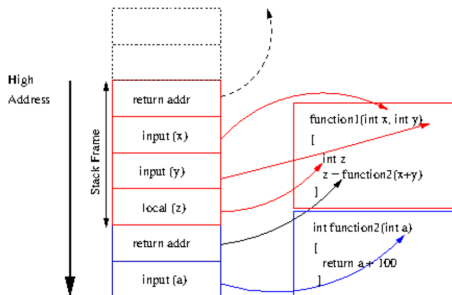
Address Space Shared among Processes



- The kernel exists **as a protected region of virtual memory within the context of every process**.
- Just like a normal user-space program, the kernel contains a code segment, global data, and a heap for dynamic memory allocation.
- Rather than having a single stack, however, **the kernel contains many stacks**;
 - For each user-space process, the kernel contains at least one stack.

What is a Stack?

- Stack is fundamental to function calls.
 - It keeps track of the functions running in a program.
 - Functions are in turn the building blocks of programs.
- Each time a function is called it gets a new **stack frame**.
 - This is an area of memory which usually contains:
 - The **address to return** to when complete;
 - The **input arguments** to the function;
 - Local variables**.

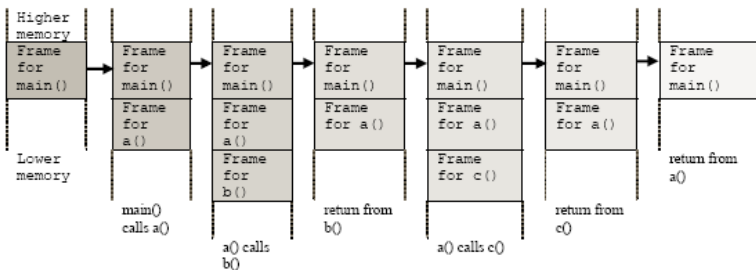


Stack (I)

```

int
c() {
    return
    0;
}
int
b() {
    return
    0;
}
int
a() {
    b();
    c();
    return
    0;
}
int
main() {
    a();
    return
    0;
}

```



Stack (II)

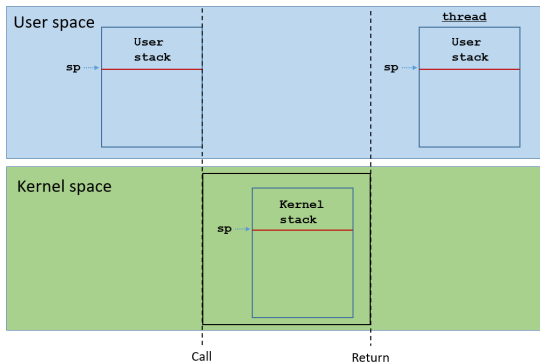
```

b:
pushl %ebp
movl %esp,%ebp
movl $0, %eax
movl %ebp,%esp
popl %ebp
ret
a:
pushl %ebp
movl %esp,%ebp
...
call b
...
movl %ebp,%esp
popl %ebp
ret

```

- The **Return address** is the address of the next instruction right after `call`
- `call <label>`
 - Push **Return address** on stack.
 - Changes the EIP register with the address represented by `label`
- `ret` instruction **pops the address from stack and set EIP register with popped value.**
 - It should be the **Return address** previously pushed by `call` instruction.

User stack vs Kernel stack (I)



- When a task (thread) runs in user-space, it uses the user-stack.
- When a system call is made, the task switches to kernel-space and starts using the kernel-stack.
- When switching back to user-space, the task resumes using the user-stack.

User stack vs Kernel stack (II)

- What's the difference between kernel stack and user stack?
 - **In short, nothing** – apart from using a different location in memory, and usually different memory access protections.
 - When executing in user mode, **kernel memory will not be accessible even if mapped.**
 - Vice versa, without explicitly being requested by the kernel code **user memory is not usually directly accessible.**
- The underlying principles are separation of privileges and security.
- A currently executing process will enter in kernel-space:
 - To execute **a system call**
 - An **interrupt** happens
 - An **exception** occurs

- 1 Operating System
- 2 Operating Systems Jargon
- 3 Types of Operating Systems
- 4 General vs Real Time Purpose
- 5 The Kernel
- 6 The Linux kernel
- 7 Execution Modes
- 8 Interfacing with hardware and software
- 9 Execution Context
- 10 Process Management
- 11 Memory Management
- 12 Dealing with time**
- 13 Other Subsystems

Time management

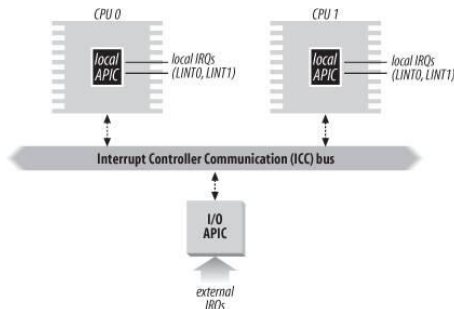
- There are a lot of kernel activities that are **time-driven**, such as balancing the processor's run-queues, process resource usage accounting, profiling, and so on;
- Time management **is based on some hardware devices that have oscillators of a known fixed frequency, counters, and comparators**;
 - Simpler devices **increment their counters** at fixed frequency, which is in some cases configurable.
 - Other devices **decrement their counters (also at fixed frequency) and are able to issue an interrupt when the counter reaches zero**.
 - More sophisticated devices **are provided with some comparison logic to compare the device counter against a specific value, and are also able to interrupt the processor when those values match**.

Time related devices (I)

- **Programmable Interval Timer (PIT)**
 - It is a counter that is able to generate an interrupt when it reaches the programmed count.
- **High Precision Event Timer (HPET)**
 - It consists of a counter and a set of comparators.
 - Each comparator has a match register and can generate an interrupt when the associated match register value matches with the main counter value;
 - The comparators can be put into one-shot mode or periodic mode.
- **Time Stamp Counter (TSC)**
 - It is a processor register that counts the number of cycles since reset;

Time related devices (II)

- Local Advanced Programmable Interrupt Controller (LAPIC), which is part of the processor chip.
 - It manages **all interrupts received by the processor either from external devices or internally generated by software**;
 - It is provided with **mechanisms for queueing, nesting, and masking interrupts**;
 - It is able **to send interrupts for external devices as well as to generate interrupts** for the respective processor.

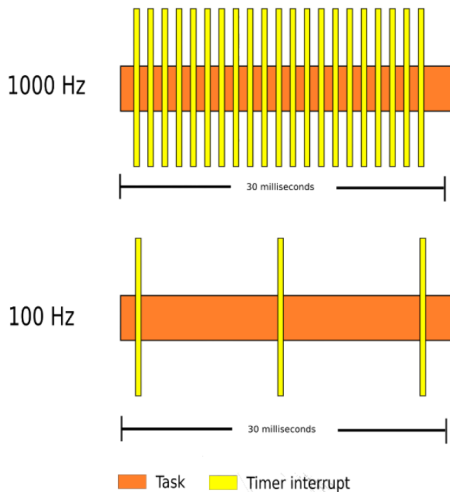


tick (I)

- Independently of the way that the time is managed, the Linux kernel **employs a periodic timer interrupt to get the control of the system**, called **tick**
 - It can be viewed as the kernel's heartbeat.
- The frequency of the this periodic timer interrupt (the tick rate) is defined by a kernel compilation option called `CONFIG_HZ`.
 - In the kernel code, this macro is only referred to by `HZ` , an acronym for hertz.

tick (II)

- Depending on the value of HZ, **the resolution of the tick** (the smallest time difference between consecutive ticks) **is more or less frequent.**



- 1 Operating System
- 2 Operating Systems Jargon
- 3 Types of Operating Systems
- 4 General vs Real Time Purpose
- 5 The Kernel
- 6 The Linux kernel
- 7 Execution Modes
- 8 Interfacing with hardware and software
- 9 Execution Context
- 10 Process Management
- 11 Memory Management
- 12 Dealing with time
- 13 Other Subsystems**

Modules

- Support for modules allows systems to have only a minimal base kernel image, with optional features and drivers supplied via loadable, separate objects.
- Modules also **enable the removal and reloading** of kernel code, facilitate debugging, and allow for the loading of new drivers on demand in response to the hot plugging of new devices.

Linux Kernel

Despite being monolithic in the sense that the whole kernel runs in a single address space, the Linux kernel is modular, supporting the dynamic insertion and removal of code from itself at runtime. Related subroutines, data, and entry and exit points are grouped together in a single binary image, a loadable kernel object, called a module.

Devices drivers

- They take on a special role in the Linux kernel.
- They **are distinct black boxes that make a particular piece of hardware respond to a well-defined internal programming interface.**
- They **hide completely the details of how the device works.**
- Typically device drivers represent physical hardware, but, not all device drivers represent physical devices.
 - Some device drivers are virtual, providing access to kernel functionality.
 - They are named pseudo devices;

Devices types

- **Block devices** are addressable in device-specified chunks called blocks and generally support seeking, the random access of data.
 - Example block devices include hard drives, Blu-ray discs, and memory devices such as flash;
- **Character devices** are generally not addressable, providing access to data only as a stream, generally of characters (bytes).
 - Example character devices include keyboards, mice, printers, and most pseudo-devices;
- **Network devices** provide access to a network (such as the Internet) via a physical adapter (such as laptop's 802.11 card) and a specific protocol (such as IP).